First Working Model Submission

Traffic Sign Detection on GTSDB

Vladislav Osetrov Andrei Ilin Matvej Shulaev Tamerlan Bagautdinov

October 2026

1 Data Loading and Preprocessing Pipeline

The first working model uses the German Traffic Sign Detection Benchmark (GTSDB). The dataset contains 900 road-scene images at a resolution of 1360×800 pixels, with bounding-box annotations covering 43 traffic-sign classes. We use the road-scene images for object detection, rather than the additional cropped sign images included in the downloaded archive.

The data-loading and preprocessing pipeline is implemented in `data/download.py` and `data/prepare.py`. The download script retrieves and extracts the archive. The preparation script reads the semicolon-separated `gt.txt` annotation file (image filename, bounding-box coordinates and class ID), checks class IDs and bounding-box validity, converts source images from PPM to RGB PNG, and generates both YOLO text labels and COCO JSON annotations. YOLO bounding boxes are expressed as normalized center coordinates and width/height; COCO annotations retain pixel-based box coordinates. This gives the project a shared set of boxes for later experiments.

The prepared dataset uses 480 training images, 120 validation images and 300 reserved test images. The train/validation partition is formed from the first 600 images using shuffled blocks of 30 adjacent image IDs, with random seed 42, to reduce leakage between neighboring frames. The 300 test images were not used for the initial metrics reported here. The split contains 712 annotated objects in training, 140 in validation and 361 in test, for a total of 1,213 objects. Images without annotated signs are retained as background examples.

2 Baseline Model and Configuration

The baseline is **YOLOv8s**, initialized with weights pretrained on COCO. Its detection output is adapted from the original 80 classes to the 43 GTSDB classes. The configured input size is 640×640 pixels, with letterbox resizing handled by Ultralytics. The model contains approximately 11.15 million parameters. Training uses random seed 42; horizontal flipping is disabled because it may change the meaning of directional traffic signs. The training recipe also uses Ultralytics' default photometric and geometric augmentations, including mosaic.

3 Training Setup

Training was performed locally using an NVIDIA GeForce RTX 5060 Ti with 16 GB of video memory, PyTorch 2.11.0+cu128 and Ultralytics 8.4.177. We used mixed-precision training, AdamW, cosine learning-rate scheduling and a batch size of 16.

Fine-tuning was carried out in two sequential phases:

- **Phase A:** freeze the first 10 YOLOv8 modules (the backbone) and train the remaining modules for 5 epochs, starting at a learning rate of 10^{-3}.
- **Phase B:** initialize from the best Phase A checkpoint, unfreeze the backbone and fine-tune the full detector with an initial learning rate of 10^{-4}. The maximum was 95 epochs, with early stopping after 20 epochs without improvement in validation fitness.

Phase B stopped after 41 epochs, with its best checkpoint recorded at epoch 21. Therefore, the run completed 5 + 41 training epochs across the two phases. Both phases save `best.pt` and `last.pt`; Phase B additionally requests periodic checkpoints every five epochs. The final baseline checkpoint is `runs/phase_b/weights/best.pt`. The Phase B training log reports approximately 0.050 hours (3 minutes); this excludes dataset download and preparation.

4 Initial Quantitative Results

We evaluate the best Phase B checkpoint on the **validation split only** (120 images, 140 annotated signs). Table 1 reports the separate Ultralytics validation run. The COCO-style export additionally produced AP of 0.340 for IoU 0.50–0.95 and AP of 0.415 at IoU 0.50; these differ slightly from the built-in metrics because the evaluation implementations/settings differ.

Table 1: Baseline validation metrics (Ultralytics evaluator).

Metric	Value
Precision	0.4176
Recall	0.4035
mAP@0.50	0.4120
mAP@0.50:0.95	0.3377

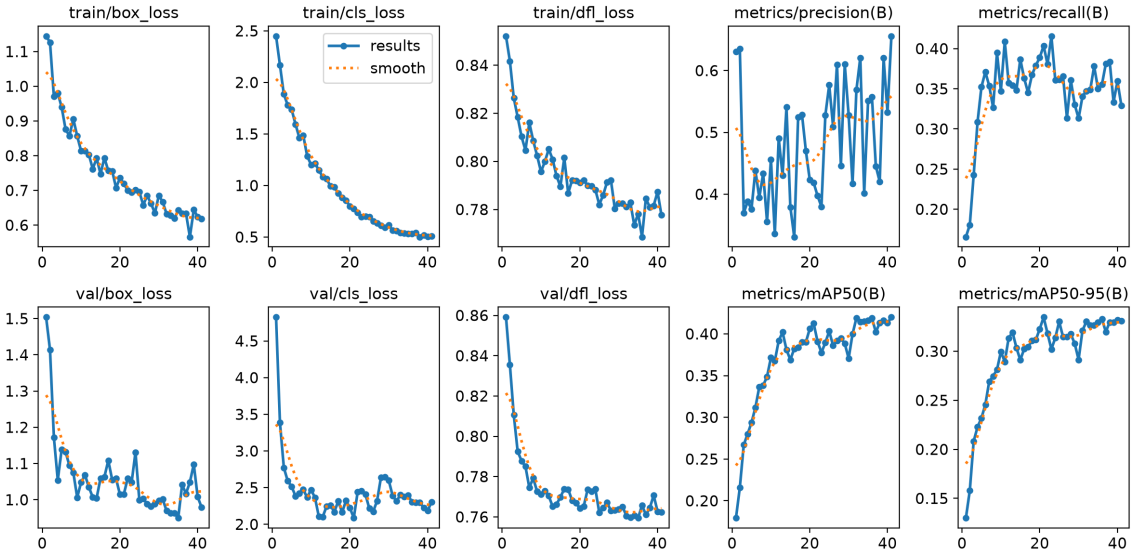


Figure 1: Phase B training losses and validation metrics across 41 epochs, exported by Ultralytics.

The validation mAP@0.50:0.95 increased markedly during the initial Phase B epochs and then plateaued. The COCO-style evaluator reported AP by object area of 0.275 for small signs, 0.510 for medium signs and 0.767 for large signs. These results suggest that detecting small traffic signs is a major limitation of the present baseline.

5 Implementation Issues and Observations

- **Class imbalance:** all 43 classes appear in training, but only 28 appear in validation. Thus, per-class metrics for rare categories are based on very few examples, and the reported aggregate validation score does not directly measure all 43 classes.
- **Duplicate annotation:** Ultralytics reported and removed one duplicate label associated with `00340.png`. The underlying annotation should be deduplicated in the preprocessing script in a subsequent code revision.
- **Small-object detection:** COCO AP by size is substantially lower for small signs than for large signs (0.275 versus 0.767). Qualitative inspection of validation predictions also shows missed signs; these examples are indicative rather than a separately quantified false-negative analysis.
- **Training behavior:** validation performance plateaued after approximately 20 Phase B epochs. Early stopping prevented unnecessary continued training and selected the best saved checkpoint rather than the final epoch.

6 Git Repository

The current project code, dataset download and preprocessing scripts, split definitions, model-training script and evaluation script are maintained at:

https://github.com/Andreyfdp/Computer_Vision_project

The repository's `requirements.txt` specifies the Python dependencies. Dataset images and model weights are stored outside version control; the trained baseline checkpoint used for the measurements above is `runs/phase_b/weights/best.pt`. To reproduce the experiment, download and prepare GTSDB using the scripts in `data/`, then run Phase A and Phase B with `src/train_yolo.py`, followed by validation with `src/evaluate.py`.